

АНАЛИЗ ИНДИВИДУАЛЬНОГО ЗАДАНИЯ

1. Изучить теоретические сведения о механизме конфигурирования устройств PCI и способам доступа к конфигурационному устройству.
2. Разработать программу формирования цикла опроса и идентификации устройств PCI, которая будет считывать два первых поля конфигурационного пространства - коды Vendor ID (производитель) и Device ID (устройство).
3. Используя файл PCI_DEVS.TXT или структуры, определенные в заголовочном файле pci_c_header.h, произвести расшифровку наименований производителей и устройств.
4. Результатом работы программы должна быть выводимая на экран или в текстовый файл таблица, содержащая следующую информацию:
 - 4.1. адрес устройства (номер шины, номер устройства и номер функции);
 - 4.2. 16-разрядный код производителя (в шестнадцатеричной системе);
 - 4.3. 16-разрядный код устройства (в шестнадцатеричной системе);
 - 4.4. производитель и название устройства.

Дополнительно выполнить индивидуальное задание в соответствии с вариантом

ВЫПОЛНЕНИЕ ЗАДАНИЯ

Листинг программы:

```
#include <stdio.h>
#include <sys/io.h>

#include <errno.h>
#include <stdlib.h>

#include <string.h>
#include <stdbool.h>
#include <values.h>
#include "pci.h"

//-----END INCLUDES-----

#define MAX_BUS 256
#define MAX_DEVICE 32
#define MAX_FUNCTIONS 8

#define ID_REGISTER 0

#define DEVICEID_SHIFT 16
#define BUS_SHIFT 16
#define DEVICE_SHIFT 11
#define FUNCTION_SHIFT 8
#define REGISTER_SHIFT 2

#define CONTROL_PORT 0x0CF8
#define DATA_PORT 0x0CFC

//-----END DEFINES-----

void PrintInfo(int bus, int device, int function);
bool IfBridge(int bus, int device, int function);
long readRegister(int bus, int device, int function, int reg);
void outputGeneralData(int bus, int device, int function, int regData);
char* getDeviceName(int vendorID, int deviceID);
char* getVendorName(int vendorID);
void outputIOMemorySpaceBARData(long regData);
void outputMemorySpaceBARData(long regData);
void outputBARsData(int bus, int device, int function);
void outputIOLimitBaseData(long regData);
void outputInterruptData(long regData);

FILE* out;

//=====12
TASK=====

void outputInterruptData(long regData) {
    fputs("        TASK 12\n", out);
    puts("        TASK 12");

    int interruptPin = (regData >> 8) & 0xFF;
    char* interruptPinData;
```

```

switch (interruptPin) {
case 0:
    interruptPinData = "not used";
    break;
case 1:
    interruptPinData = "INTA#";
    break;
case 2:
    interruptPinData = "INTB#";
    break;
case 3:
    interruptPinData = "INTC#";
    break;
case 4:
    interruptPinData = "INTD#";
    break;
default:
    interruptPinData = "invalid pin number";
    break;
}

fprintf(out, "Interrupt pin: %s\n", interruptPinData);
printf("Interrupt pin: %s\n", interruptPinData);


int interruptLine = regData & 0xFF;

fputs("Interrupt line: ", out);
printf("Interrupt line: ");
if (interruptLine == 0xFF) {
    fputs("unused/unknown input\n", out);
    printf("unused/unknown input\n");
}
else if (interruptLine < 16) {
    fprintf(out, "%s%d\n", "IRQ", interruptLine);
    printf("%s%d\n", "IRQ", interruptLine);
}
else {
    fputs("invalid line number\n", out);
    puts("invalid line number\n");
}

}

//=====9
TASK=====

void outputIOLimitBaseData(long regData) {
    unsigned reg1Data = regData;
    unsigned reg2Data = reg1Data >> 8;
    fputs("TASK 9\n", out);
    puts("TASK 9");
    long IOBase = reg1Data & 0xFF;
    long IOLimit = (reg2Data & 0xFF);

```

```

if (IOBase == 0) {
    fprintf(out, "I/O Base: 0x00\n");
    printf("I/O Base: 0x00\n");
}
else {
    fprintf(out, "I/O Base: %#lx\n", IOBase);
    printf("I/O Base: %#lx\n", IOBase);
}

if (IOLimit == 0) {

    fprintf(out, "I/O Limit: 0x00\n");
    printf("I/O Limit: 0x00\n");
}
else {

    fprintf(out, "I/O Limit: %#lx\n", IOLimit);
    printf("I/O Limit: %#lx\n", IOLimit);
}
}
//=====2
TASK=====

void outputBARsData(int bus, int device, int function) {
    fputs("TASK 2\n", out);
    puts("TASK 2");
    fputs("Base Address Registers:\n", out);
    puts("Base Address Registers:\n");
    int i;
    for (i = 0; i < 6; i++) {
        fprintf(out, "\tRegister %d: ", i);
        printf("\tRegister %d: ", i);
        long regData = readRegister(bus, device, function, 4 + i);
        if (regData) {
            if (regData & 1) {
                outputIOMemorySpaceBARData(regData);
            }
            else {
                outputMemorySpaceBARData(regData);
            }
        }
        else {

            fprintf(out, "0x0000, ");
            printf("0x0000, ");
            fputs("unused register\n", out);
            puts("unused register");
        }
    }
}

void outputMemorySpaceBARData(long regData) {
    unsigned long reg1Data = regData >> 4;
    fprintf(out, "%#lx, ", reg1Data);
    printf("%#lx, ", reg1Data);
    fputs("memory space register", out);
    printf("memory space register");
    int typeBits = (regData >> 1) & 3;

    switch (typeBits) {
    case 0:
        fprintf(out, " (any position in 32 bit address space)\n");
        printf(" (any position in 32 bit address space)\n");
        break;

```

```

        case 1:
            fprintf(out, "(below 1MB)\n");
            printf("(below 1MB)\n");
            break;
        case 2:
            fprintf(out, "(any position in 64 bit address space)\n");
            printf("(any position in 64 bit address space)\n");
            break;
        default:
            fprintf(out, "(reserved)\n");
            printf("(reserved)\n");
            break;
    }
}

void outputIOMemorySpaceBARData(long regData) {
    unsigned long reg1Data = regData - 1;
    fprintf(out, "%#lx, ", reg1Data);
    fputs("I/O space register\n", out);
    printf("%#lx, ", reg1Data);
    printf("I/O space register\n");
}

//=====GENERAL=====
=====

char* getVendorName(int vendorID) {
    int i;
    for (i = 0; i < PCI_VENTABLE_LEN; i++) {
        if (PciVenTable[i].VendorId == vendorID) {
            return PciVenTable[i].VendorName;
        }
    }
    return NULL;
}

char* getDeviceName(int vendorID, int deviceID) {
    int i;
    for (i = 0; i < PCI_DEVTABLE_LEN; i++) {
        if (PciDevTable[i].VendorId == vendorID && PciDevTable[i].DeviceId ==
deviceID) {
            return PciDevTable[i].DeviceName;
        }
    }
    return NULL;
}

void outputVendorData(int vendorID)
{
    char* vendorName = getVendorName(vendorID);
    fprintf(out, "Vendor ID: %04d, %s\n", vendorID, vendorName ? vendorName :
"unknown vendor");
    printf("Vendor ID: %04d, %s\n", vendorID, vendorName ? vendorName : "Unknown
vendor");
}

void outputDeviceData(int vendorID, int deviceID)
{
    char* deviceName = getDeviceName(vendorID, deviceID);
    fprintf(out, "Device ID: %04d, %s\n", deviceID, deviceName ? deviceName :
"unknown device");
}

```

```

    printf("Device ID: %04d, %s\n", deviceID, deviceName ? deviceName : "Unknown
device");
}

void outputGeneralData(int bus, int device, int function, int regData) {
    fprintf(out, "%x:%x:%x\n", bus, device, function);
    printf("%x:%x:%x\n", bus, device, function);
    int deviceID = regData >> DEVICEID_SHIFT;
    int vendorID = regData & 0xFFFF;
    outputVendorData(vendorID);
    outputDeviceData(vendorID, deviceID);
}

long readRegister(int bus, int device, int function, int reg) {
    long configRegAddress = (1 << 31) | (bus << BUS_SHIFT) | (device <<
DEVICE_SHIFT) | (function << FUNCTION_SHIFT) | (reg << REGISTER_SHIFT);
    outl(configRegAddress, CONTROL_PORT);
    return inl(DATA_PORT);

    return 0;
}

bool IfBridge(int bus, int device, int function) {
    long htypeRegData = readRegister(bus, device, function, 3);
    return ((htypeRegData >> 16) & 0xFF) & 1;
}

void PrintInfo(int bus, int device, int function) {
    long idRegData = readRegister(bus, device, function, ID_REGISTER);

    if (idRegData != 0xFFFFFFFF) { // if there is a device
        outputGeneralData(bus, device, function, idRegData);

        if (IfBridge(bus, device, function)) {
            fprintf(out, "\nIs bridge\n\n");
            printf("\nIs bridge\n\n");
            outputIOLimitBaseData(readRegister(bus, device, function, 7));
            outputInterruptData(readRegister(bus, device, function, 15));

        }
        else {
            fprintf(out, "\nNot a bridge\n\n");
            printf("\nNot a bridge\n\n");
            outputBARsData(bus, device, function);

        }
        fputs("-----\n", out);
        puts("-----\n");
    }
}

int main() {

    if (iopl(3)) {
        printf("I/O Privilege level change error: %s\nTry running under ROOT
user\n", strerror(errno));
    }
}

```

```

        return 2;
    }

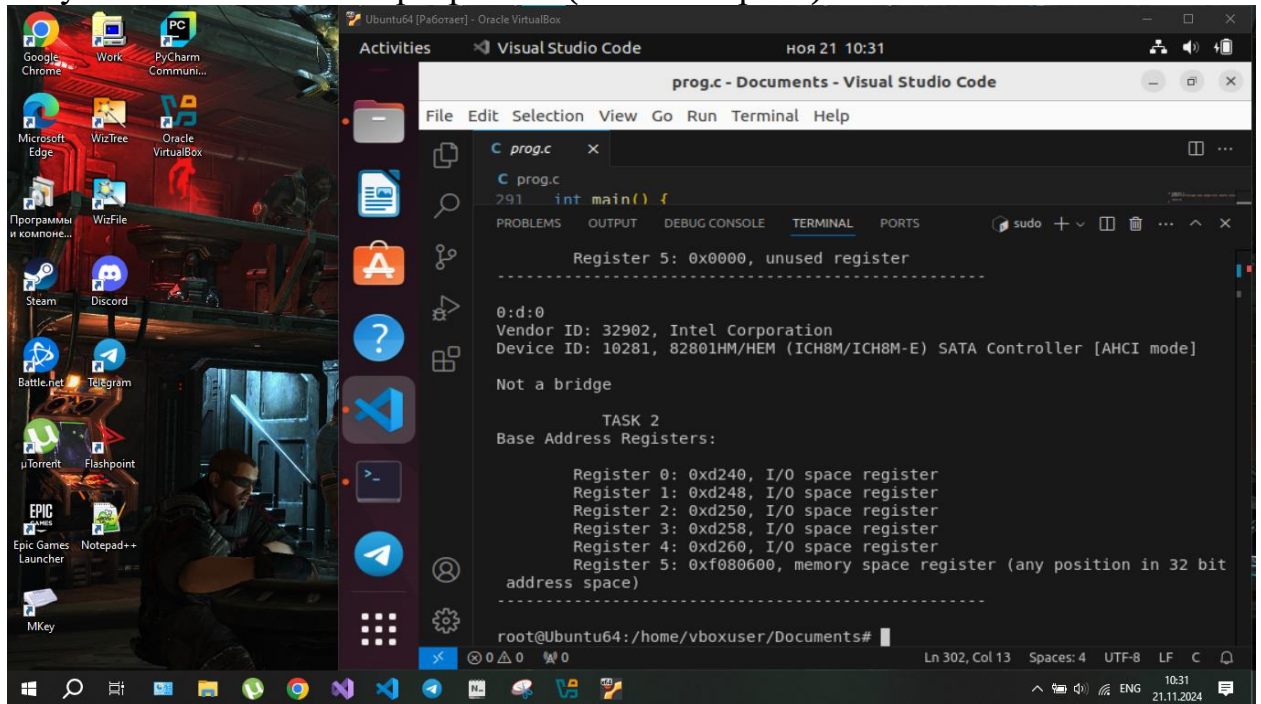
    int bus;
    int device;
    int func;
    out = fopen("output.txt", "w");

    for (bus = 0; bus < MAX_BUS; bus++) {
        for (device = 0; device < MAX_DEVICE; device++) {
            for (func = 0; func < MAX_FUNCTIONS; func++) {
                PrintInfo(bus, device, func);
            }
        }
    }

    fclose(out);
    return 0;
}

```

Результат выполнения программы (снимок экрана):



ЗАКЛЮЧЕНИЕ

В ходе выполнения лабораторной работы были изучены теоретические сведения о механизме конфигурирования устройств PCI и способам доступа к конфигурационному устройству, реализовано программное средство, удовлетворяющее поставленной задаче.